GUIA TECNICA – Aprendiendo C# con Charp (ACC).

Autores:

- German Uriel Evangelista Martinez
- Aldo Juan Figueroa Espinoza

Asesores:

Asesor técnico – Francisco Javier Tafolla Granados

1.Índice.

Contenido

1.Índice. 2
2.Introducción. 2
3.Software usado. 3
4.Hardware usado. 4
5.Requerimientos Técnicos para Recrear o Construir ACC (para desarrolladores) 5
6.Requerimientos para su Uso como Usuario Final 6
7.Arquitectura del proyecto. 8
8.Patrón de Arquitectura. 12
9.Herramientas usadas. 14
10.Metodología usada. 15
11.Manual de estilos para la creación de lecciones en ACC. 16
12.Manual Técnico-Pedagógico para la Creación de Lecciones en ACC. 20
13.Manual de Inserción de Lecciones ACC 25

2.Introducción.

"Aprendiendo C# con Charp (ACC)" es una plataforma educativa interactiva desarrollada con tecnologías modernas del ecosistema .NET, cuyo objetivo principal es facilitar el aprendizaje del lenguaje de programación C# de forma didáctica, estructurada y progresiva.

La plataforma está pensada para estudiantes, autodidactas y docentes que buscan una herramienta sólida y atractiva para comprender los fundamentos y aplicaciones prácticas de C#. ACC no solo permite visualizar contenido teórico y ejemplos prácticos, sino que también ofrece ejercicios interactivos, un sistema de evaluación, un asistente de inteligencia artificial personalizado llamado Charp, y funcionalidades como calendario, seguimiento de progreso y aulas virtuales.

El desarrollo de ACC fue llevado a cabo bajo una planificación metodológica intensiva, con una arquitectura modular, escalable y adaptable a múltiples dispositivos. Se utilizaron herramientas de código abierto, tecnologías modernas como Blazor, Entity Framework Core, Redis, Docker y servicios personalizados que permiten su integración eficiente en diversos entornos.

Esta guía técnica tiene como finalidad documentar el proceso de construcción, herramientas empleadas, decisiones técnicas clave y estructura interna de la plataforma, con el objetivo de servir como referencia para desarrolladores que deseen replicar, mantener o escalar el sistema.

3.Software usado.

Durante el desarrollo de **Aprendiendo C# con Charp (ACC)**, se utilizó un conjunto robusto de herramientas y plataformas modernas del ecosistema .NET y del desarrollo web. A continuación, se describen los componentes principales del entorno de desarrollo:

Software	**Versión utilizada**	**Propósito**
.NET SDK	8.0	Desarrollo backend y frontend con Blazor.
Visual Studio	2022 community	Entorno de desarrollo integrado principal.
SQL Server	2022 Express	Motor de base de datos relacional para el almacenamiento de datos.
Docker Desktop	Última estable	Orquestación de servicios, contenedores de base de datos y Redis.
Redis	7.x	Caché de datos para mejorar el rendimiento y respuesta de la API.
Postman	10.x	Pruebas de endpoints y validación de servicios REST.
Git	2.x	Control de versiones y gestión de ramas del proyecto.
GitHub	N/A	Repositorio remoto para colaboración y control del código fuente.
Aspire (Microsoft)	Preview	Observabilidad, monitoreo y configuración de servicios distribuidos.
Entity Framework Core	8.0	ORM para el acceso a datos y generación de migraciones.
AutoMapper	Última estable	Mapeo automático entre entidades y DTOs.
xUnit + Moq	Última estable	Pruebas unitarias y simulación de dependencias.
Serilog	Última estable	Sistema de logs y trazabilidad del sistema.

4.Hardware usado.

El desarrollo de **Aprendiendo C# con Charp (ACC)** fue llevado a cabo por dos desarrolladores principales, utilizando equipos de gama media-alta general que demostraron ser suficientes para las tareas de programación, pruebas y despliegue de la plataforma

Integrante	**Dispositivo**	**Especificaciones**
Germán	ASUS TUF Gaming F17	- **Procesador:** Intel Core i5-11400H (6 núcleos, hasta 4.5 GHz) - **Memoria RAM:** 32 GB DDR4 - **Almacenamiento:** SSD NVMe de 1 TB - **Tarjeta Gráfica:** NVIDIA GeForce RTX 2050 (4GB GDDR6) - **Sistema Operativo:** Windows 11 Home - **Pantalla:** 17.3" FHD (1920x1080), 144 Hz
Aldo	Apple MacBook Air (M1, 2020)	- **Procesador:** Apple M1 (8 núcleos: 4 de rendimiento y 4 de eficiencia) - **Memoria RAM:** 8 GB unificada - **Almacenamiento:** SSD de 256 GB - **Sistema Operativo:** macOS Big Sur (actualizable) - **Pantalla:** 13.3" Retina (2560x1600)

5.Requerimientos Técnicos para Recrear o Construir ACC (para desarrolladores)

Esta sección detalla los requerimientos necesarios para que un desarrollador pueda clonar, compilar, ejecutar y mantener la solución completa de *Aprendiendo C# con Charp (ACC)* en un entorno local o de desarrollo profesional. El entorno está basado en el ecosistema .NET, contenedores, y herramientas modernas de integración y despliegue.

Hardware:

Componente	**Requerido (mínimo)**	**Recomendado**
Procesador (CPU)	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / Ryzen 7 o superior
Memoria RAM	8 GB	16 GB o más
Almacenamiento	SSD con 20 GB libres	SSD NVMe con 50+ GB libres
Pantalla	Resolución 1366x768	Full HD (1920x1080) o superior

Software:

Herramienta/Plataforma	**Versión**	**Propósito**
.NET SDK	8.0	Compilación y ejecución de proyectos en la solución.
Visual Studio Community 2022	Última estable	Entorno de desarrollo con soporte Blazor y EF.
SQL Server Express	2022	Base de datos principal del sistema
Docker Desktop	Última estable	Orquestación de servicios y contenedores.
Redis (opcional)	7.x	Almacenamiento en caché.
Git	2.x o superior	Control de versiones y gestión de código.
Postman	10.x	Pruebas de endpoints y validación de APIs.
Aspire CLI	Preview	Observabilidad y manejo de servicios distribuidos.

Nota: Las versiones listadas son las utilizadas durante el desarrollo. Se recomienda mantenerlas actualizadas y validar compatibilidades en cada actualización mayor.

6.Requerimientos para su Uso como Usuario Final

Esta sección describe los requerimientos mínimos y recomendados para que cualquier usuario pueda utilizar correctamente la plataforma Aprendiendo C# con Charp (ACC) desde un navegador web o dispositivo compatible, sin necesidad de instalar software adicional.

Requisitos de navegador web.

Elemento	**Recomendado**
Navegadores compatibles	Google Chrome, Microsoft Edge, Firefox, Safari (últimas versiones)
Compatibilidad	Soporte para JavaScript, WebAssembly (WASM), y cookies
Resolución de pantalla	Mínima: 1024x768 / Recomendada: 1366x768 o superior
Conectividad	Conexión estable a Internet (mínimo 5 Mbps)
Seguridad	HTTPS habilitado, navegador actualizado, cookies activas

Compatibilidad con dispositivos.

Dispositivo	**Estado**	**Observaciones**
Computadora de escritorio o portátil	Compatible	Versión completa con experiencia completa de usuario
Tablet (Android/iPad)	Compatible	Interfaz adaptada, sin pérdida de funcionalidades
Smartphone (Android/iOS)	Compatible	Interfaz responsive con navegación optimizada para móvil

ACC fue desarrollado usando Blazor, lo que permite compatibilidad total con dispositivos modernos y adaptación automática a pantallas más pequeñas sin necesidad de aplicaciones móviles nativas.

Consideraciones adicionales.

- El rendimiento puede verse afectado en dispositivos antiguos con poca memoria o procesadores lentos.
- Se recomienda mantener el navegador actualizado para evitar errores de renderizado o incompatibilidades con funcionalidades modernas.

7.Arquitectura del proyecto.

ACC-Complex ¿Qué es?

Es la solución de Aprendiendo C# con Charp (ACC) la cual está organizada en 11 proyectos distribuidos en distintas carpetas lógicas según su rol en la arquitectura. A continuación, se describen:

Carpeta – Src

Proyecto	**Función**
ACC.API	Proyecto central del backend. Expone controladores REST para el manejo de módulos, lecciones, aulas, tareas, etc.
ACC.AppHost	Punto de entrada orquestador. Usa Aspire para levantar, configurar y conectar todos los servicios.
ACC.Data	Contiene las entidades del dominio, DbContext, migraciones y configuración de acceso a la base de datos.
ACC.ExternalClients	Consume APIs externas (como Charp-IA). Se encarga de encapsular la lógica de integración.
ACC.ServiceDefaults	Biblioteca compartida para todos los microservicios. Agrega descubrimiento de servicios, resiliencia, health checks y OpenTelemetry.
ACC.Shared	Contiene DTOs, interfaces, enums y tipos comunes como ServiceResult<T>. Es la base de la comunicación entre capas.
API_CompilerACC	Servicio especializado que compila código C# en tiempo real usando Roslyn. Devuelve errores de compilación o resultado exitoso.

Carpeta – tests

Proyecto	**Función**
ACC.Tests	Contiene pruebas unitarias usando xUnit y Moq. Se enfoca en servicios, validaciones y controladores.

Proyectos Frontend(A excepción de ACC.WebApp, que funge como backend):

Proyecto	**Función**
ACC.WebApp	Backend de autenticación y sistema de cuentas. Maneja login, registro, roles, autorización, tokens, etc.
ACC.WebApp.Client	Aplicación cliente basada en **Blazor WebAssembly**. Consume tanto ACC.WebApp como ACC.API vía HTTP.
ACC.MultiPlataform	Aplicación **MAUI Blazor App** orientada a dispositivos móviles, tablets y escritorio. Proporciona una experiencia nativa, con enfoque en movilidad.

¿Cómo interactúan los proyectos? – Flujo del ACC-Complex
Se presenta un ejemplo de cómo interactúan los proyectos en la solución.

Inicio de Sesión (Autenticación con ACC.WebApp)

1. El usuario ingresa desde **ACC.WebApp.Client** (Blazor WebAssembly).
2. El cliente envía sus credenciales a **ACC.WebApp** vía HTTPS.
3. ACC.WebApp:
    - Valida credenciales con Identity.
    - Asigna roles (Estudiante, Docente, Admin).
    - Devuelve un token o contexto de sesión.
4. El token es guardado en el navegador para futuras llamadas a **ACC.API**.

Carga de Lecciones y Contenido (Educativo vía ACC.API)

1. **ACC.WebApp.Client** utiliza el token autenticado y solicita a **ACC.API**:
    - Lecciones, módulos, subtemas, prácticas, evaluaciones...
2. **ACC.API** consulta su base de datos mediante **ACC.Data.**
3. La respuesta se envía como DTOs desde **ACC.Shared**.

Compilación de Código (API_CompilerACC)

1. En una lección práctica, el usuario escribe código C#.
2. **ACC.WebApp.Client** envía el código al endpoint de **API_CompilerACC.**
3. Este lo compila usando Roslyn y responde:
    - Éxito con salida estándar.
    - Error con mensaje detallado.

App Nativa (ACC.MultiPlataform)

1. Funciona como cliente alterno, orientado a móviles, usando MAUI.
2. Se conecta a **ACC.WebApp** (autenticación) y a **ACC.API** (contenido).
3. Se inicia la app desde una pantalla embebida del login de ACC.WebApp, manteniendo centralizada la autenticación.

Telemetría y Comunicación (ACC.ServiceDefaults)

- Todos los servicios (API, Compiler, WebApp, etc.) utilizan **ACC.ServiceDefaults.**
- Esto agrega:
    - OpenTelemetry (para trazas, logs y métricas)
    - Descubrimiento automático de servicios
    - Resiliencia en llamadas HTTP
    - Health checks automáticos (Healthy, Unhealthy, Exited)

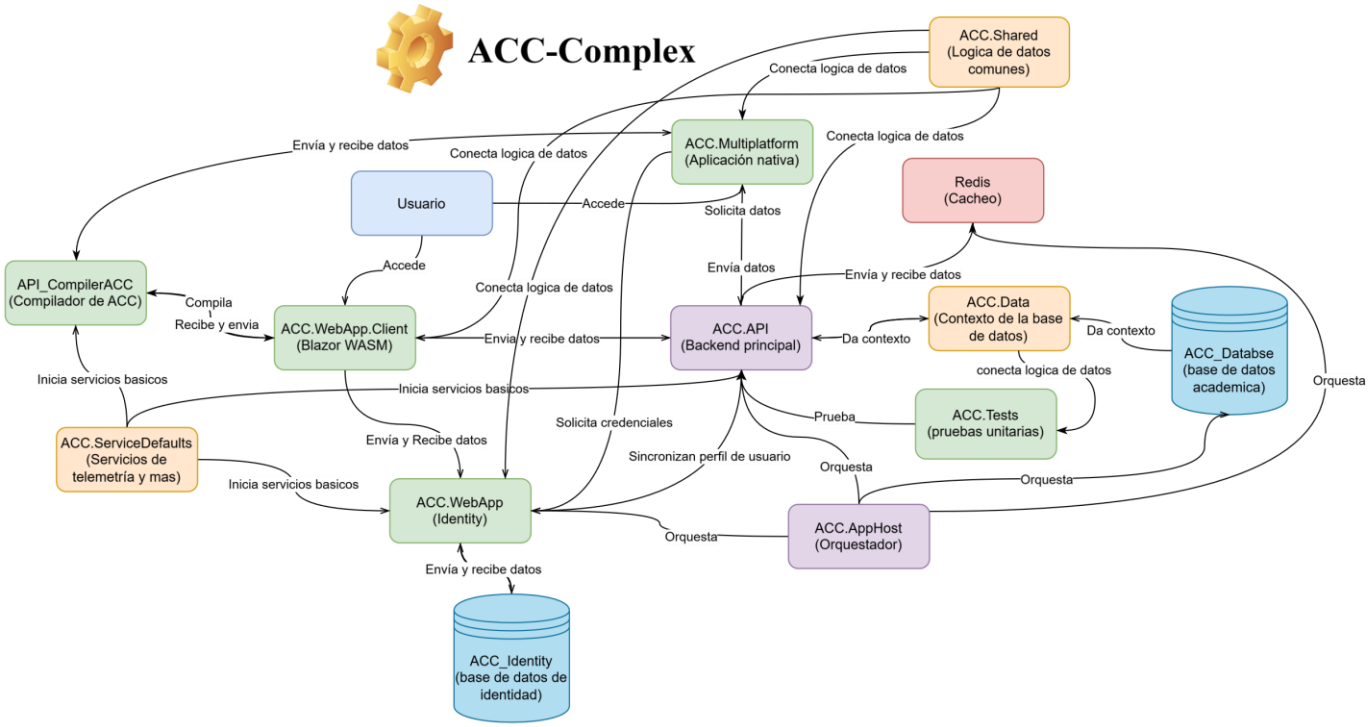
Servicios Externos (Charp-IA)

1. **ACC.WebApp.Client** envía preguntas a Chatbase (servicio IA pre entrenado).
2. Recibe respuestas filtradas, orientativas, nunca respuestas de código directo.

Pruebas (ACC.Tests)

- Simula el comportamiento de servicios usando Moq.
- Valida escenarios como:
    - Carga de lecciones inválidas.
    - Compilación con errores.
    - Roles no autorizados accediendo a recursos.

Y a continuación, un diagrama de como es esta arquitectura al completo:



8.Patrón de Arquitectura.

La solución Aprendiendo C# con Charp (ACC) se diseñó siguiendo un enfoque modular distribuido, fundamentado en principios de Clean Architecture, combinado con una infraestructura moderna basada en .NET Aspire para facilitar la resiliencia, el monitoreo y el descubrimiento de servicios.

Principios aplicados:

Separación de responsabilidades

Cada componente tiene una responsabilidad única y clara:

- ACC.WebApp: Autenticación y gestión de cuentas.
- ACC.API: Gestión del contenido educativo.
- API_CompilerACC: Compilación de código.
- ACC.WebApp.Client: Interfaz de usuario.
- ACC.MultiPlataform: Experiencia móvil y nativa.

Independencia de capas

Las capas se comunican únicamente a través de contratos definidos (DTOs, interfaces de ACC.Shared), lo que permite cambiar una implementación sin afectar las demás.

Inversión de dependencias

Los servicios internos dependen de abstracciones y no de implementaciones concretas, facilitando pruebas unitarias y mantenimiento.

Reutilización y extensión

Los servicios comunes (como ACC.ServiceDefaults) se referencian desde todos los microservicios, reduciendo duplicación de código y asegurando consistencia en resiliencia, telemetría y descubrimiento.

Capas lógicas en la arquitectura ACC-Complex

Capa	Responsabilidad
Presentación	ACC.WebApp.Client, ACC.MultiPlataform. Interfaz directa con el usuario final.
Aplicación	ACC.API, API_CompilerACC, ACC.WebApp. Lógica de negocio, seguridad, flujos.
Infraestructura	ACC.Data, ACC.ExternalClients, ACC.ServiceDefaults. Base de datos, APIs externas, resiliencia.
Dominio compartido	ACC.Shared. Tipos comunes, contratos, DTOs, reglas base.

Modelo distribuido con orquestación Aspire

Gracias a Aspire, los servicios se:

Ejecutan independientemente, se descubren automáticamente entre sí (via service discovery), reportan su estado y métricas (con OpenTelemetry), se reinician o escalan fácilmente con resiliencia activa y se prueban y despliegan con Docker como contenedores orquestados.

Patrones adicionales usados:

- JWT / Cookie Auth (según entorno y plataforma).
- DTO Mapping con AutoMapper.
- Validación con FluentValidation.
- Tests aislados con xUnit + Moq.
- OpenTelemetry + Health Checks para observabilidad.
- Compilación dinámica con Roslyn en API_CompilerACC.

9.Herramientas usadas.

Durante el desarrollo de Aprendiendo C# con Charp (ACC) se integraron herramientas modernas, librerías de alto rendimiento y utilidades enfocadas en acelerar el desarrollo, mejorar la calidad del código y mantener la trazabilidad del sistema.

A continuación, se listan las más relevantes clasificadas por propósito:

Herramientas de desarrollo

Herramienta	**Propósito**
Visual Studio Community 2022	Entorno principal de desarrollo con soporte para .NET 8, MAUI y Blazor.
Docker Desktop	Contenerización de servicios y bases de datos, integración con Aspire.
SQL Server Management Studio (SSMS)	Administración de bases de datos SQL Server.
Git + GitHub	Control de versiones, ramas y colaboración.
Postman	Pruebas de endpoints HTTP y simulación de escenarios API.
.NET Aspire CLI	Observabilidad, descubrimiento de servicios, configuración compartida.

Librerías utilizadas en el backend

Librería	**Propósito**
ASP.NET Core	Framework base para construcción de servicios web y APIs REST.
Entity Framework Core	ORM para acceso a base de datos y migraciones.
AutoMapper	Mapeo automático entre entidades y DTOs.
FluentValidation	Validación robusta de modelos y entradas de usuario.
xUnit	Framework de pruebas unitarias.
Moq	Simulación de dependencias en pruebas.
Serilog	Registro estructurado de logs.

OpenTelemetry	Trazabilidad y métricas distribuidas en todos los servicios.
Roslyn (.NET Compiler Platform)	Compilación de código C# en tiempo real (en API_CompilerACC).

Librerías utilizadas en el Frontend

Librería/tecnología	**Propósito**
Blazor WebAssembly	Aplicación SPA que se ejecuta completamente en el navegador.
MAUI Blazor	Plataforma nativa para escritorio y móvil (usada en ACC.MultiPlataform).
Bootstrap	Estilización y componentes visuales (por confirmar si usas alguno).
JSInterop	Comunicación entre C# y JavaScript cuando es necesario.

Integraciones externas

Servicio	**Uso**
Redis	Almacenamiento en caché distribuido.
Chatbase (Charp IA)	Asistente inteligente para orientación educativa dentro de ACC.

10.Metodología usada.

El desarrollo de Aprendiendo C# con Charp (ACC) se llevó a cabo mediante una estrategia metodológica flexible, orientada a resultados, que combinó elementos de metodologías ágiles con planificación iterativa, pruebas constantes y objetivos semanales.

Se opto entonces por SCRUM una metodología de desarrollo de software ágil, la cual consiste en iteraciones llamadas sprints, organización con los asesores los cuales asumieron los roles de scrum máster y product owner. Los autores fungieron como Development Team.

11.Manual de estilos para la creación de lecciones en ACC.

Objetivo:

Guardar coherencia entre todas las lecciones para mantener un estilo visual general, simplificar estilos CSS y darle una identidad propia a la plataforma.

A continuación, se detallan los estilos:

Estructura Base

Clase: .leccion-container
Contenedor general para las lecciones educativas.

- Centrado automáticamente (margin: auto)
- Ancho máximo: 1200px
- Fuente base: Arial, sans-serif

Secciones Internas

Clases: .leccion-teoria, .leccion-ejemplos, .leccion-practicas
Dividen claramente el contenido en partes fundamentales de la lección.

- Fondo: #16213e
- Borde lateral izquierdo: #3a3a67
- Bordes redondeados y sombra suave

Tipografía y Texto

Títulos

- Fuente: Nunito
- Color: mediumpurple
- Peso: negrita

Párrafos y texto general

- Color: #d1d1e0
- Tamaño: 1rem
- Altura de línea: 1.6

Listas

- Color: blanco
- Viñetas con disc

Tablas

- Fondo general: #16213e, con bordes visibles
- Encabezado: #1f4068 con texto blanco (#f8f8ff)

Bloques de Código

Uso recomendado únicamente para fragmentos medianos o grandes.

- Fondo: #1E1E2F
- Texto blanco
- Scroll horizontal si se desborda
- Bordes redondeados
- Sombra interna

Botones

- Fondo: #3a3a67
- Hover: #4f5d7a
- Bordes redondeados
- Sin borde exterior
- Texto blanco

Imágenes

- Tamaño máximo: 100% del contenedor
- Redondeo: 8px

Texto Destacado

Clase: .highlight

- Color: #ffcc00
- Texto en negrita

Alertas y Notificaciones

Las alertas se usan para enfatizar mensajes clave dentro de cualquier sección. Se clasifican por colores según su propósito.

Clase	Uso recomendado	Borde lateral
.alert-info	Tips, notas extra, referencias útiles	Azul claro #4fb3ff
.alert-success	Buenas prácticas o logros	Verde #6fffbf
.alert-warning	Advertencias sobre malentendidos comunes	Amarillo #ffc14f
.alert-error	Errores importantes o fallos críticos	Rojo fuerte #ff4f4f

---

Fomentador (Recuadro de Apoyo Visual)

Clase: .fomentador
Caja destacada para redirección a materiales externos o guías ampliadas.

- Fondo: #2a2a3b
- Texto: #e0e0e0
- Enlaces:
    - Color: #4fb3ff
    - Hover: #6fc1ff

Iframes Responsivos

Clase: .responsive-iframe-no-modal

- Ancho y alto: 100%
- Sin bordes

Fuentes Personalizadas

Nunito (Títulos)

font-family: 'Nunito', sans-serif;

Graduate (Opcional para títulos decorativos)

font-family: 'Graduate', serif;

Estas fuentes son cargadas automáticamente mediante @font-face

12.Manual Técnico-Pedagógico para la Creación de Lecciones en ACC.

Objetivo:

Establecer un estándar pedagógico y técnico para la creación de lecciones en la plataforma educativa **ACC (Aprendiendo C# con Charp)**, asegurando la calidad del contenido, la coherencia didáctica y el cumplimiento de los principios de progresividad, claridad y aprendizaje significativo.

Estructura General de una Lección

Cada lección debe seguir la siguiente estructura general, adaptándose según el nivel y el contexto del subtema. No todas las secciones son obligatorias, pero se recomienda mantener la secuencia cuando se apliquen:

Orden	Sección	Descripción
1	Teoría	Presenta el concepto. Puede usar lenguaje simple en niveles bajos, pero es válido emplear términos técnicos y precisos cuando el nivel del usuario lo permita.
2	Ejemplos	Casos buenos y malos, con explicaciones claras de cada uno. Pueden incluirse otros tipos de ejemplos si aportan valor, siempre que sean claros y bien explicados.
3	Práctica guiada	Paso a paso del concepto en acción. No se recomienda usar el compilador en cada paso.
4	Actividad	Reto o ejercicio principal. Aquí se puede usar el compilador de ACC.
5	Evaluación	Evaluación automática (si aplica).
6	Fomentador	Recursos adicionales, dando prioridad a los de la biblioteca ACC.

Recomendaciones Pedagógicas

- Adaptar el lenguaje al contexto: Usar lenguaje claro para principiantes, pero no simplificar en exceso en niveles avanzados. Ser directo es clave en módulos superiores.
- Una idea por párrafo: Evitar saturar la mente del usuario con múltiples ideas en el mismo bloque.

- Foco en el aprendizaje activo: Toda lección debe motivar al usuario a aplicar o reflexionar.
- Gamificación equilibrada: Utilizar elementos visuales y refuerzos positivos, pero sin exagerar.
- Aplicar la Taxonomía de Bloom: Introducir, aplicar y evaluar. Usar verbos orientados a cada nivel (definir, aplicar, analizar, construir, etc.).

Guía por Sección:

1. **Teoría**
   - Máximo cinco párrafos sugeridos (flexible).
   - Adaptar el nivel de complejidad según el módulo.
   - Utilizar alertas informativas para definiciones o aclaraciones clave.
2. **Ejemplos**
   - Incluir siempre un buen ejemplo y, si aplica, un mal ejemplo claramente identificado.
   - Explicar el porqué del resultado.
3. **Práctica Guiada**
   - Presentar un ejercicio dividido en pasos, explicando qué se está haciendo y por qué.
   - No usar el compilador en cada paso, para evitar saturación visual.
   - Especialmente útil en lecciones encadenadas (como proyectos).
4. **Actividad**
   - Ejercicio final que el usuario debe resolver por sí mismo.
   - Aquí sí debe incluirse el compilador si es relevante.
   - Debe estar alineada con los objetivos de la lección.
5. **Evaluación**
   - Aún no se encuentra en uso activo, pero se recomienda definir al menos una pregunta de selección múltiple o de análisis para el futuro.
   - Las evaluaciones estarán integradas automáticamente cuando la plataforma lo permita.
6. **Fomentador**
   - Caja visual para recomendar recursos opcionales.
   - Priorizar recursos de la Biblioteca ACC.
   - Se pueden incluir enlaces externos solo si el contenido no está cubierto internamente.
   - Si no se conoce el enlace, dejar marcado como # para completar más adelante.

Criterios de validación de una leccion

Criterio	¿Obligatorio?	Peso
Cumple con estructura base	Sí	30%
Claridad y progresividad	Sí	20%
Tiene actividad	Sí	20%
Incluye ejemplos significativos	Sí	15%
Usa reforzadores visuales	Opcional	10%
Contiene recurso adicional	Opcional	5%

Plantilla Recomendada para las Lecciones

```html
<div class="leccion-container">

  <div class="leccion-teoria">
    <h3>Título del Concepto</h3>
    <p>Explicación de la teoría...</p>
  </div>

  <div class="leccion-ejemplos">
    <h3>Ejemplos</h3>
    <pre><code>codigo de ejemplo: Console.WriteLine("hola ACC");</code></pre>
    <div class="alert alert-error">Este sería un mal ejemplo porque...</div>
  </div>

  <div class="leccion-practicas">
    <h3>Práctica guiada</h3>
    <p>Paso 1...</p>
    <p>Paso 2...</p>
  </div>

  <iframe class="responsive-iframe-no-modal" src="URL-de-un-video-cualquiera"></iframe>

  <div class="fomentador">
    ¿Quieres aprender más? <a href="#">Consulta la biblioteca</a>
  </div>

</div>
```

Cópiala a continuación si es que la necesitas:

```
<div class="leccion-container">
  <div class="leccion-teoria">
    <h3>Título del Concepto</h3>
    <p>Explicación de la teoría...</p>
  </div>
  <div class="leccion-ejemplos">
    <h3>Ejemplos</h3>
    <pre><code>código de ejemplo: Console.WriteLine("hola ACC");</code></pre>
    <div class="alert alert-error">Este sería un mal ejemplo porque...</div>
  </div>
  <div class="leccion-practicas">
    <h3>Práctica guiada</h3>
    <p>Paso 1...</p>
    <p>Paso 2...</p>
  </div>
  <iframe class="responsive-iframe-no-modal" src="URL-de-un-video-cualquiera"></iframe>
  <div class="fomentador">
    ¿Quieres aprender más? <a href="#">Consulta la biblioteca</a>
  </div>
</div>
```

Nota: El iframe debe usarse únicamente para incluir recursos opcionales (como videos, simuladores o visualizaciones).

Este manual es un documento vivo y puede evolucionar junto con la plataforma ACC. Toda propuesta de mejora debe respetar el enfoque pedagógico principal: **enseñar C# de forma clara, progresiva y motivadora para cualquier nivel de usuario**.

13.Manual de Inserción de Lecciones ACC

Objetivo: Explicar cómo insertar lecciones en el sistema ACC utilizando SQL, respetando la estructura modular y dinámica basada en el orden de secciones.

Estructura Base de la Tabla Lecciones

La tabla debe contener los siguientes campos:

Campo	Tipo de Dato	Descripción
IdLeccion	INT (Identity, PK)	Identificador único de la lección (clave primaria).
TituloLeccion	NVARCHAR	Título visible de la lección.
DescripcionLeccion	NVARCHAR	Breve descripción del contenido de la lección.
ContenidoLeccion	NVARCHAR(MAX)	Contenido en HTML enriquecido que será renderizado.
TieneActividad	BIT	Indica si la lección incluye una actividad externa.
UrlActividad	NVARCHAR(MAX)	URL del recurso o actividad interactiva (por ejemplo, Educaplay).
TieneEvaluacion	BIT	Indica si se tiene asociada una evaluación (no se usa actualmente).
IdEvaluacion	INT (FK opcional)	Identificador de la evaluación asociada, si aplica.
TieneCompilador	BIT	Indica si la lección activa el compilador interactivo.
IdSubTema	INT (NOT NULL)	Relación con el subtema al que pertenece esta lección.
OrdenSecciones	NVARCHAR(MAX)	Arreglo JSON como texto que define el orden en que se muestran secciones.

Ejemplo de INSERT para una Lección Completa

```
INSERT INTO Lecciones (
    TituloLeccion,
    DescripcionLeccion,
    ContenidoLeccion,
    TieneActividad,
    UrlActividad,
    TieneEvaluacion,
    IdEvaluacion,
    TieneCompilador,
    IdSubTema,
    OrdenSecciones
)
VALUES (
    'Introducción al diseño de software',
    'Se te presentarán el diseño de software, su importancia y más',
    '<p>En esta lección aprenderás...</p><iframe width="100%" height="315" src="https://www.youtube.com/embed/xyz123" frameborder="0" allowfullscreen></iframe>',
    1,
    'https://es.educaplay.com/juego/123456-software_basico.html',
    1,
    5,
    1,
    1002,
    '["html", "actividad", "compilador", "evaluacion"]'
);
```

<u>Nota importante:</u>

- OrdenSecciones debe ser un arreglo JSON válido, codificado como texto.
- El campo TieneEvaluacion no se utilizará por ahora, pero debe mantenerse como parte del esquema para compatibilidad futura.

Lista de Etiquetas Disponibles en OrdenSecciones

"html" – Renderiza el contenido HTML de la lección

"actividad" – Muestra el componente de actividad externa desde UrlActividad

"compilador" – Activa el compilador interactivo de ACC

"evaluacion" – Habilita el botón de acceso a evaluación (si aplica)

Recomendaciones para Autores de Contenido

1. Utiliza etiquetas HTML estándar (<h2>, <p>, <ul>, <iframe>) dentro de ContenidoLeccion.
2. Los elementos <iframe> se renderizan directamente desde el HTML cargado.
3. Mantén el orden lógico de las secciones según el flujo pedagógico deseado.
4. Si alguna sección está desactivada (por ejemplo, TieneActividad = 0), no será renderizada en el frontend.

Resultado en el Frontend

El sistema frontend interpretará dinámicamente el contenido del campo OrdenSecciones y renderizará las secciones activas en el orden definido.

Por ejemplo, si nuestro arreglo de secciones es este:

[“html”, “actividad”, “compilador”]

El resultado visible será:

1. Sección de contenido HTML
2. Actividad interactiva desde el enlace configurado
3. Compilador para que el usuario aplique lo aprendido

Este sistema garantiza flexibilidad, modularidad y una experiencia de aprendizaje efectiva para todos los usuarios de **Aprendiendo C# con Charp (ACC)**.